

```

"""
===== OOPS (Object Oriented Programming) =====
Here we create classes (Blueprint) and on the basis of class we create objects.

class => It is like a Blueprint for Objects
object => real entity based on class.

^ NOTE :- Class is a user-defined data Type.

=> A class is a collection of different properties(attributes/variables) and
methods(functions) , that work together.

syntax :-
-----

class className:
    properties/attributes/variables
    methods / functions

objectName = className()

!=== Parameters for Object Oriented programming languages ===!
=> constructor
=> Inheritance
=> Polymorphism
=> Encapsulation
=> Abstraction
"""

class car:
    name="Mahindra"
    Model = "Mahendra 2.0"

    def start(self):
        print("Car starts")
    def stop(self):
        print("Car Stop")

    def showInfo(self,name,model,Launch_year=2023):
        print("The car name is : ",name)
        print("Model of car : ",model)
        print("launching Year : ",Launch_year)
        print()
        self.start()

car1=car()
car2=car()

```

```

# car1.showInfo("Maruti","Maruti 1200")
# car2.showInfo("Honda","Honda 812",2025)

# print(car1.name)
# print(car1.Model)
# car1.start()
# car1.stop()
# print(type(car1))

# l=[1,2,3,4,5]
# l.append(10)
# print(l)
# print(type(l))

# ^ ===== constructor in python oops =====
# Constructor is a function inside the class,It calls automatically when we create #
the object of the class.
"""
def __init__(self):
    statements...
"""

class dog:
    # this is constructor function
    def __init__(self,name,age,breed):
        self.Dog_Name = name
        self.Dog_Age = age
        self.Dog_Breed = breed

    def show_info(self):
        print("Dog Name : ",self.Dog_Name)
        print("Dog Age : ",self.Dog_Age)
        print("Dog Breed :",self.Dog_Breed)

# dog1=dog("Buddy",5,"German Shepard")
# dog1.show_info()

#& ===== Inheritance in python oops =====
# when a class used methods and properties of another class is called inheritance.

# class Parent:
#     f_name=input("Enter Father's Name : ")
#     m_name=input("Enter Mother's Name : ")

#     def parent(self):
#         print("This is from parent Class")

```

```

#     def parent_intro(self):
#         print("The Father's Name is : ",self.f_name)
#         print("The Mother's Name is : ",self.m_name)
#         print("-----\n")

# p1 = Parent()
# p1.parent_intro()

# class Child(Parent):
#     c_name = input("Enter Child Name : ")

#     def child_info(self):
#         print("The Name of child is : ",self.c_name)

# child1 = Child()

# child1.parent_intro()
# child1.child_info()

#! ===== Polymorphism =====
# when a property or method which have same name in different class , but different
behavior is called polymorphism.

class cat:
    def sound(self):
        print("Meow-Meow")

class dog:
    def sound(self):
        print("Bark")

class lion:
    def sound(self):
        print("Roar")

# cat1=cat()
# dog1=dog()
# lion1=lion()

# cat1.sound() => Meow-Meow
# dog1.sound() => Bark
# lion1.sound() => Roar

#* ===== Encapsulation =====
# used to restrict, direct use of some properties and methods.

#^ Access Modifiers in a class :-
#^ -----

```

```

"""
1. public :- Can be accessed inside or outside the class.
2. protected :- Can not directly access, access by any method or by inheritance
ex :- _variableName/_functionName
3. private :- Can only used by inheritance.
ex :- __variableName/__functionName

"""

class user:
    name = "Ram"
    age = 21
    __bankBalance = 10

    def __showDetails(self):
        print(f"""
        Name : {self.name}
        Age : {self.age}
        Balance : {self.__bankBalance}
        """)

class user2(user):
    def data(self):
        self._user__showDetails() # name mangling to access private method

# user_2 = user2()
# user_2.data()

#* ===== Abstraction in OOPS =====
# Abstraction is used to hide complex implementation, show necessary information.
# When a class is an abstract ,methods and properties defined in the abstract class,
# a derived class have to follow all of them.

"""
for abstraction : we have to import abc module.
-----
from abc import ABC,abstractmethod
"""
from abc import ABC , abstractmethod

class collage(ABC):
    def event(self):
        print("This is an event function")

    @abstractmethod
    def attendance(self,atnds):
        pass

```

```

class student(collage):
    def __init__(self,name,stream,entry_year):
        self.name=name
        self.stream = stream
        self.entry_year = entry_year

    print(f"""
===== student's Data =====
        Name of Student : {name}
        -----
        Stream of user : {stream}
        -----
        Admission Year of Student : {entry_year}
        -----
=====
""")
    def attendance(self, atnds):
        self.attend = atnds

        if(self.attend<75):
            print('Your Attandance is not Sufficeint\nSo, can not be appear in Exam
!!!!')
        else:
            print("You are eligible for exams.")

student1 = student("Ram","B.Tech(computer Science)",2023)
student1.attandance(9)

```